FCEFyN
FACULTAD DE CIENCIAS EXACTAS, FÍSICAS y NATURALES

UNC Universidad Nacional de Córdoba

09807 - Redes de Computadoras

Trabajo Práctico Nº 6

Taller de armado y verificación de cables Cat5/Cat5e bajo estándar T568A/B

WireGuardians
Viberti, Benjamin[1]
Espinoza Sutta, Aaron Alejandro[1]
Cleri, Juan Ignacio[1]
Pineda, Juan Ignacio[1]
Grafión, Atilio Leonel[1]
Badenes, Tomás[1]
Oviedo, Ignacio Nicolas[1]
Mendez, Jorge Nicolas[1]

23/10/2026

[1]{b.viberti,aaron.espinoza_4500,ignacio.cleri,juan.ignacio.pineda,atilio.grafion,tomasbadenes,ignacio.oviedo.239,jorge.mendez}@mi.unc.edu.ar

***Objetivo* — Tomar contacto con elementos de la capa física y ganar experiencia en armado y verificación de conexiones de red.**

Parte 1 — Taller de armado y verificación de cables Cat5/Cat5e bajo estándar T568A/B

Consigna 1

Investigar los pasos para crimpar un cable bajo la norma T568A/B DERECHO. No cruzado.

Consigna 3

VERIFICAR la buena construcción de un cable. Para ello, buscar otro grupo e intercambiar cables.

Punto a)

Inspección visual para la calidad constructiva (sean extremadamente críticos 😈) tomen fotos, indiquen que ven bien y que ven mal. Justifiquen.

Consigna 4

Documentar con que grupo intercambiaron cables (pueden intercambiar más de una vez, no importa, mientras no tengan su propio cable). Tomar fotografías que evidencien el proceso técnico de verificación.

Parte 2 — Equipamiento físico, verificación y utilización de equipos de red y generación de tráfico

En esta parte armamos una LAN con dos computadoras de nuestro grupo, a las que llamamos PC A y PC B, conectadas a un switch Cisco Catalyst 2950 del laboratorio, verificamos la conectividad con `ping` y generamos tráfico UDP con `ncat`, que capturamos con Wireshark. Todo el material (fotos, capturas de pantalla y archivos `.pcapng`) está en la carpeta `Experimento/`.

Levantar y testear una LAN

Consigna 1

Configurar MANUALMENTE la interfaz de red de la computadora que elija el grupo para realizar la actividad.

Configuramos a mano la interfaz cableada de las dos computadoras del grupo (la PC B usó un adaptador USB–Ethernet Realtek, que Linux nombra `enp3s0f3u1u1`):

Tabla 1. Direccionamiento usado. Las MAC salen de las respuestas ARP capturadas.

PC	Dirección IPv4	Máscara	Gateway	MAC
A (servidor / emisor)	`192.168.10.2`	`255.255.255.0` (/24)	`192.168.10.1`	`50:54:7b:9f:f9:68`
B (cliente / receptor)	`192.168.10.3`	`255.255.255.0` (/24)	`192.168.10.1`	`00:e0:4c:68:0e:87`

En la PC A configuramos la IP desde la terminal con `nmcli`, la herramienta de línea de comandos de NetworkManager:

```
nmcli con add type ethernet ifname enp4s0f3u2c2 con-name tp-manual \
  ipv4.method manual ipv4.addresses 192.168.10.2/24 ipv4.gateway 192.168.10.1
```

- `con add type ethernet`: crea un perfil de conexión nuevo para una interfaz cableada.
- `ifname enp4s0f3u2c2`: la interfaz de red a la que se aplica el perfil.
- `con-name tp-manual`: el nombre del perfil. Al tenerlo separado, después se puede volver a la conexión automática sin perder la configuración original.
- `ipv4.method manual`: desactiva DHCP. La dirección se fija a mano (puntos a, b y c de la consigna).
- `ipv4.addresses 192.168.10.2/24`: la dirección IPv4 y la máscara en notación CIDR (`/24` equivale a `255.255.255.0`).
- `ipv4.gateway 192.168.10.1`: el gateway por defecto.

En la PC B usamos el mismo comando, cambiando la interfaz por la suya (el adaptador USB–Ethernet `enp3s0f3u1u1`) y la dirección por `192.168.10.3`:

```
nmcli con add type ethernet ifname enp3s0f3u1u1 con-name tp-manual \
  ipv4.method manual ipv4.addresses 192.168.10.3/24 ipv4.gateway 192.168.10.1
```

Punto d)

Justificar las direcciones elegidas y si al final tenían o no sentido para la actividad.

- **Direcciones IP:** elegimos el rango privado `192.168.0.0/16` (RFC 1918), que no se enruta en Internet y no choca con nada externo. Usamos la subred `192.168.10.0/24` en vez de la `192.168.1.0/24` del ejemplo de la consigna para no pisarnos con otros grupos ni con el switch, que según el CDP tiene la IP de administración `192.168.1.2`.
- **Máscara /24:** deja las dos PCs en la misma subred, así se comunican directo por el switch, sin router. Sobran direcciones (254 hosts), pero es la máscara más simple de configurar. Tuvo sentido: `ping`, `ncat` y HTTP funcionaron.
- **Gateway `192.168.10.1`:** lo pusimos por convención (primera dirección de la subred), pero **no tenía sentido**: en la LAN no hay ningún router. Las capturas lo confirman: en `captura-http-404-y-clase2.pcapng` (tramas 19–37) las dos PCs preguntan `Who has 192.168.10.1?` por ARP varias

veces y nadie responde. Esas consultas aparecen cuando el sistema intenta salir de la subred (por ejemplo, a Internet). Para hablar entre `.2` y `.3` el gateway nunca se usa, porque el destino está en la misma subred.

Consigna 2

De a dos o más grupos por switch, conectarán una computadora por cada grupo al mismo utilizando los cables que tengan de la parte 1 de este TP. Verificar que pueden llegar a la PC de otro grupo usando ping.

La consigna propone conectar al switch una PC de cada grupo. En nuestro caso, las dos computadoras (PC A y PC B) eran de nuestro grupo, así que la prueba de conectividad se hizo entre ellas. Esto nos permitió ver los dos extremos de cada prueba (emisor y receptor) y capturar tanto lo que se enviaba como lo que llegaba.

Conectamos las dos notebooks al switch con cables UTP (Figura 1). Ninguna de las dos tiene puerto Ethernet, así que usamos adaptadores USB–RJ45.

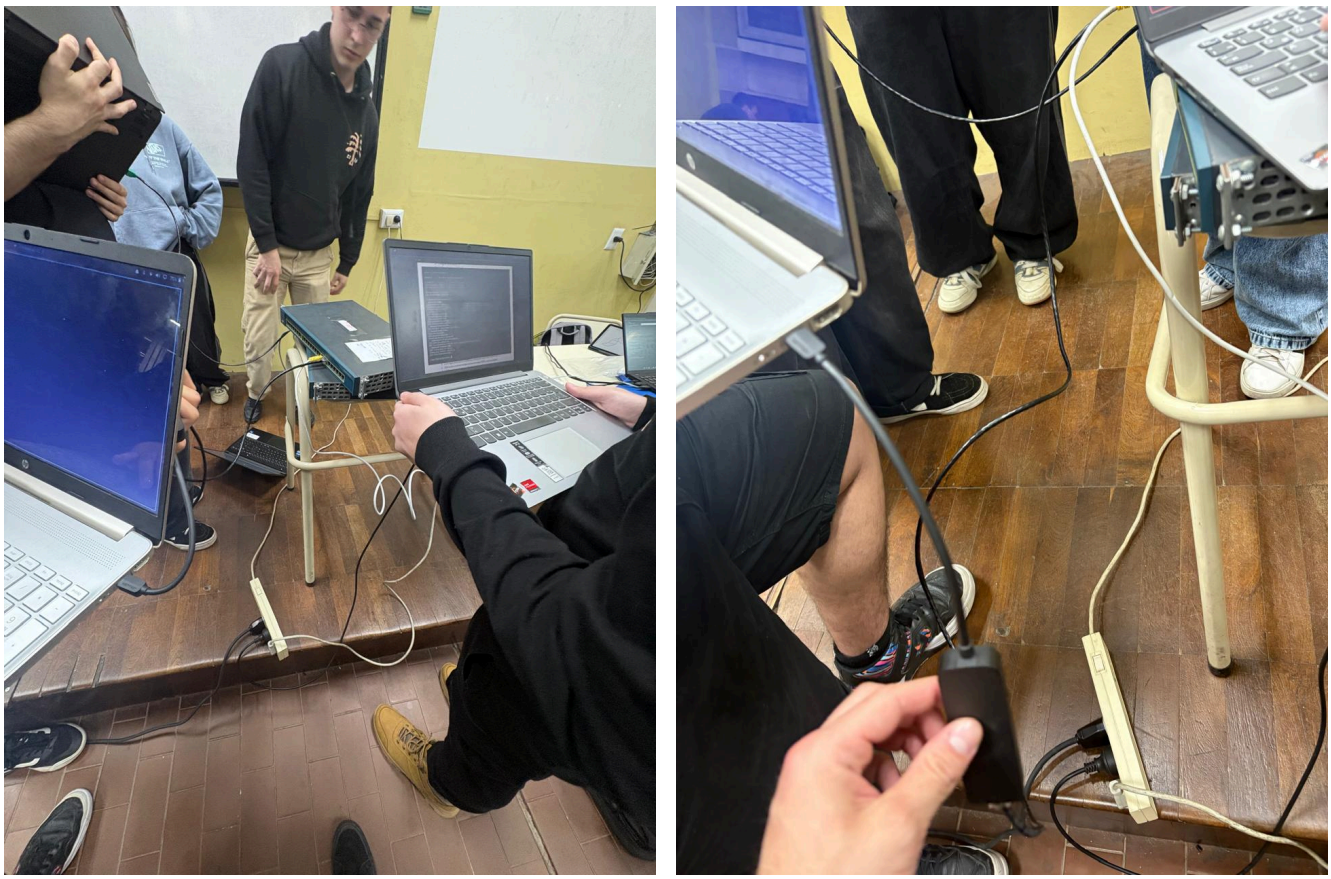


Figura 1. Montaje: las notebooks conectadas al switch (izquierda) y el adaptador USB–Ethernet de la PC B (derecha).

Desde la PC B (`192.168.10.3`) hicimos `ping 192.168.10.2`:

```
22 packets transmitted, 22 received, 0% packet loss, time 21262ms
rtt min/avg/max/mdev = 0.439/0.890/1.358/0.289 ms
```

No se perdió ningún paquete y el RTT promedio fue de 0,89 ms, coherente con dos equipos conectados por un solo switch. El TTL de las respuestas es 64, el valor inicial de Linux, sin decrementar: no hay ningún router en el medio.



Figura 2. La PC B con el resultado del `ping` y luego el receptor `ncat` en la terminal; atrás, otra notebook apoyada sobre el switch.

Consigna 3

Utilizando ncat, establecer un receptor UDP en una computadora y, desde la segunda computadora, enviar mensajes al receptor.

- En la PC B (receptor) ejecutamos `ncat -u -l 5000`: queda escuchando datagramas UDP en el puerto 5000.
- En la PC A (emisor) ejecutamos `ncat -u 192.168.10.3 5000` y escribimos varias líneas (`hola`, `hola hola`, `jhjkhjk`, `k`, líneas vacías…).

Cada línea escrita en el emisor apareció en el receptor (Figura 3). En el medio cortamos el receptor con Ctrl+C y lo volvimos a lanzar.

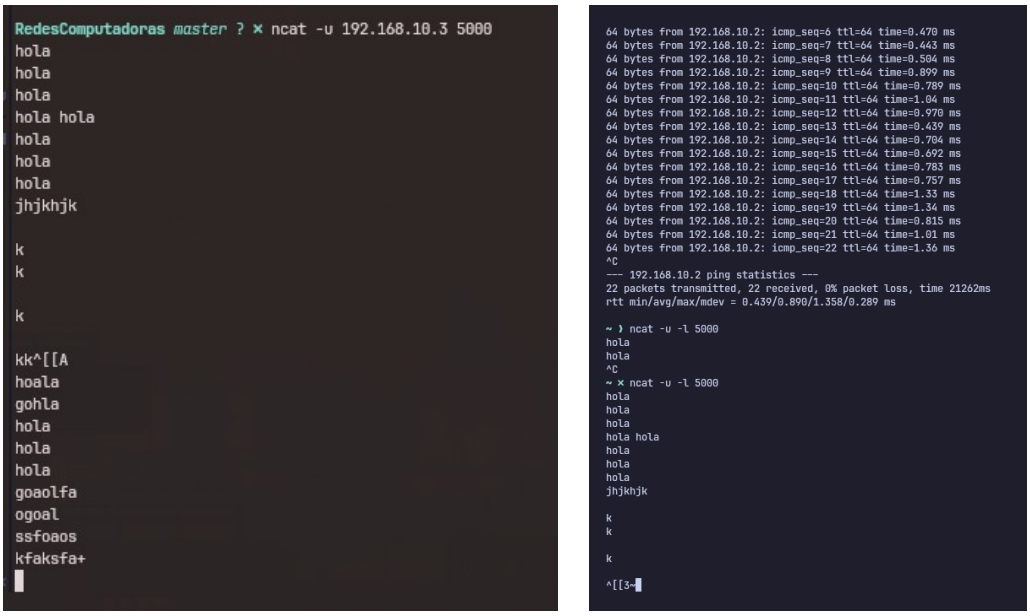


Figura 3. A la izquierda, el emisor (PC A) enviando mensajes. A la derecha, el receptor (PC B): el final del `ping` y los mensajes recibidos por `ncat`.

Consigna 4

Capturar los paquetes recibidos desde la computadora del otro grupo con wireshark. Documentar.

Como las dos PCs eran nuestras, en lugar de la computadora de otro grupo el emisor fue la PC A. Capturamos con Wireshark en la interfaz `enp3s0f3u1u1` de la PC B mientras recibía los mensajes enviados por la PC A (archivo `captura-udp-ncat.pcapng`). En 84 segundos se capturaron 70 tramas, pero solo 12 son los datagramas UDP de los mensajes. El resto es tráfico de control que genera el propio switch:

Tabla 2. Contenido de `captura-udp-ncat.pcapng` por protocolo.

Protocolo	Tramas	Qué es
UDP	12	Los mensajes de `ncat` (`192.168.10.2:49792` → `192.168.10.3:5000`).
STP	43	BPDUs de Spanning Tree que el switch manda cada 2 s.
LOOP	9	*Keepalive* de Cisco que el switch se manda a sí mismo cada 10 s para detectar bucles.
DTP	3	*Dynamic Trunking Protocol*: el switch ofrece negociar un enlace troncal (cada 30 s).
CDP	1	El switch anuncia su modelo, IOS y puerto (ver Opcional 1).
ARP	2	La PC A pregunta la MAC de `192.168.10.3` y la PC B responde.

Con el filtro `udp` quedan los 12 datagramas (Figura 4). Decodificando el campo de datos de cada uno se reconstruyen los mensajes:

Tabla 3. Datagramas UDP capturados. La longitud UDP incluye los 8 bytes del encabezado.

Trama	Tiempo [s]	Datos	Long. UDP [B]
4	3,08	`hola\n`	13
11	13,87	`hola hola\n`	18
13	15,39	`hola\n`	13
15	16,98	`hola\n`	13
17	18,49	`hola\n`	13
27	28,73	`jhjkhjk\n`	16
28	28,95	`\n`	9
29	29,21	`k\n`	10
30	29,40	`k\n`	10
31	29,56	`\n`	9
32	29,73	`k\n`	10
33	29,86	`\n`	9

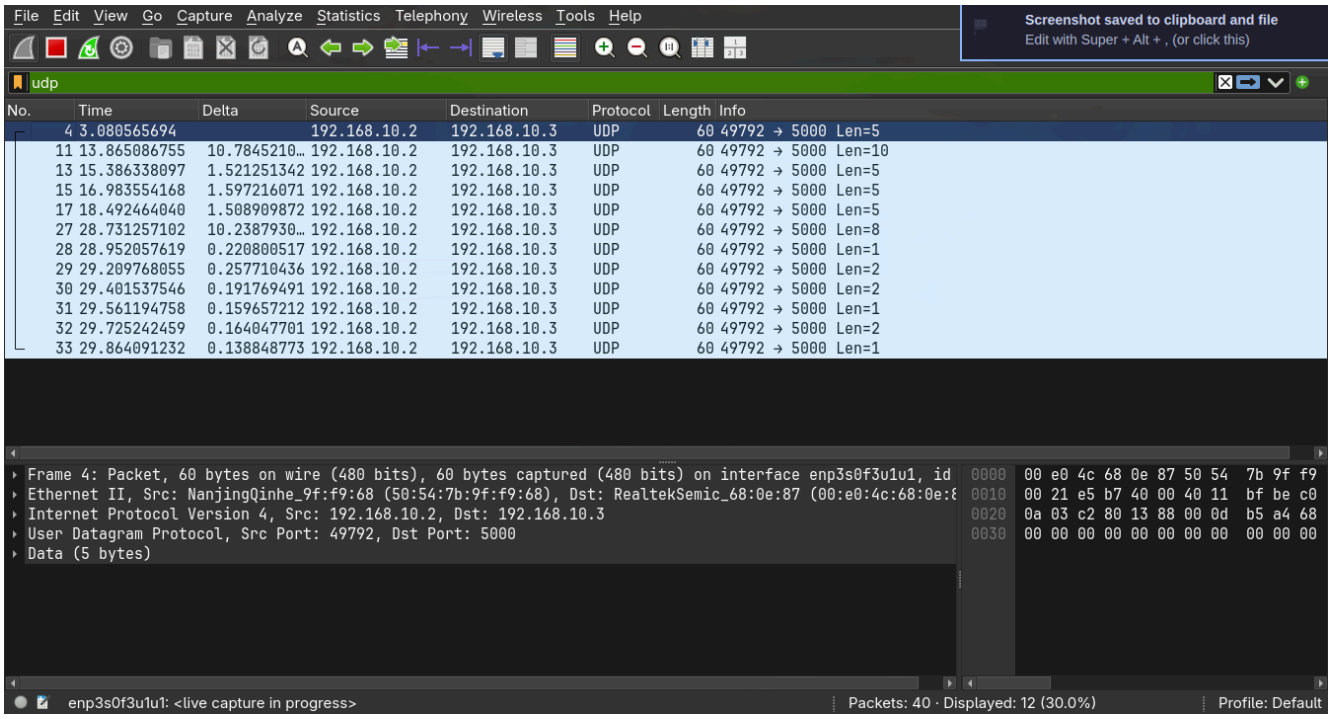


Figura 4. Wireshark con el filtro `udp`: la lista de todos los mensajes del experimento y, abajo, el detalle del primer datagrama recibido.

Lo que se observa:

- **Un datagrama por línea:** `ncat` envía cada línea apenas se presiona Enter, con el `\n` incluido. Una línea vacía genera un datagrama de 1 byte de datos.
- **Encabezado UDP mínimo:** solo tiene puerto origen, puerto destino, longitud y checksum (8 bytes). En la trama 4: 8 bytes de encabezado + 5 de datos (`hola\n`) = 13 bytes.
- **Puerto origen fijo:** todos los datagramas salen del puerto efímero 49792, porque `ncat` usa un único socket para toda la sesión. El destino es siempre el 5000.
- **Relleno Ethernet:** todas las tramas miden 60 bytes aunque el contenido real ocupe 47 (14 Ethernet + 20 IP + 8 UDP + 5 datos). Ethernet exige un mínimo de 64 bytes con el FCS, así que la placa completa con ceros (se ven en el volcado hexadecimal de la Figura 4).

- **Una sola dirección:** todo el tráfico UDP va de `.2` a `.3`. La PC B nunca contesta nada a nivel de transporte.

Consigna 5

Punto a)

¿Se observa algún procedimiento de establecimiento de conexión antes de enviar los datos?

Punto b)

¿Existe algún mecanismo de confirmación de recepción equivalente a los ACK de TCP?

Punto c)

¿Qué ocurre si se envían mensajes mientras el receptor UDP no está ejecutándose?

Punto d)

¿UDP garantiza que los mensajes lleguen en el mismo orden en que fueron enviados?

Punto e)

¿Qué aplicaciones reales utilizan UDP y por qué?

Parte 3 — Desarrollar un pequeño servidor

Usamos la misma LAN de la Parte 2, con las dos computadoras de nuestro grupo: la PC A (`192.168.10.2`) hizo de servidor y la PC B (`192.168.10.3`) de cliente.

Consigna 1

En una de las computadoras, crear un directorio que contenga al menos tres archivos de diferentes tipos (por ejemplo, TXT, PDF e imagen). Iniciar un servidor HTTP utilizando Python:

```
python3 -m http.server 8000
```

En la PC A lanzamos `python3 -m http.server 8000` dentro del repositorio de la materia, que ya tiene archivos de varios como: Markdown (`README.md`, `CLAUDE.md`), PDF (el programa de la materia, en `Material/`), texto plano (apuntes de clase `.txt`) e imágenes. En las respuestas capturadas se identifica con el encabezado `Server: SimpleHTTP/0.6 Python/3.14.7`.

Consigna 2

Desde la segunda computadora, acceder al servidor mediante un navegador web, utilizando la dirección IP correspondiente y el puerto 8000 (o el que hayas elegido). Descargar uno de los archivos disponibles.

Desde la PC B abrimos `http://192.168.10.2:8000/` en el navegador: (Figura 5).

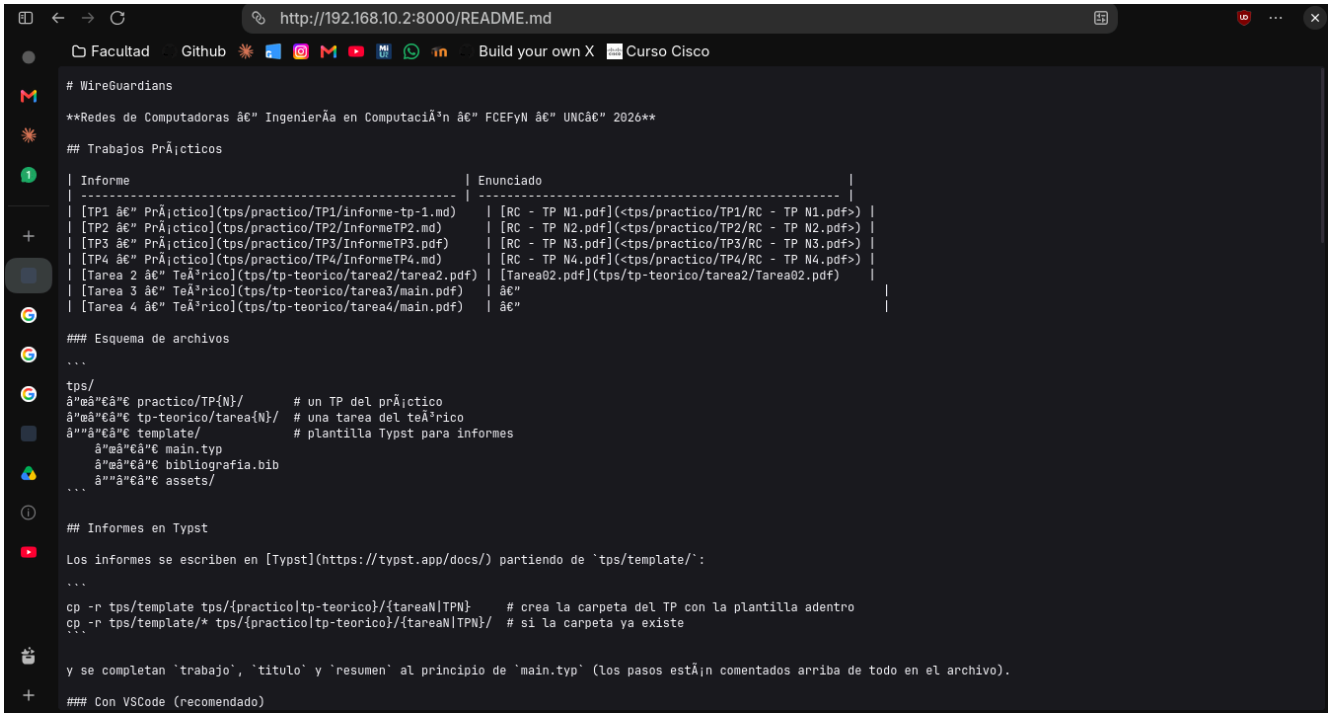


Figura 5. Archivo README.md por la PC A vistos desde el navegador de la PC B.

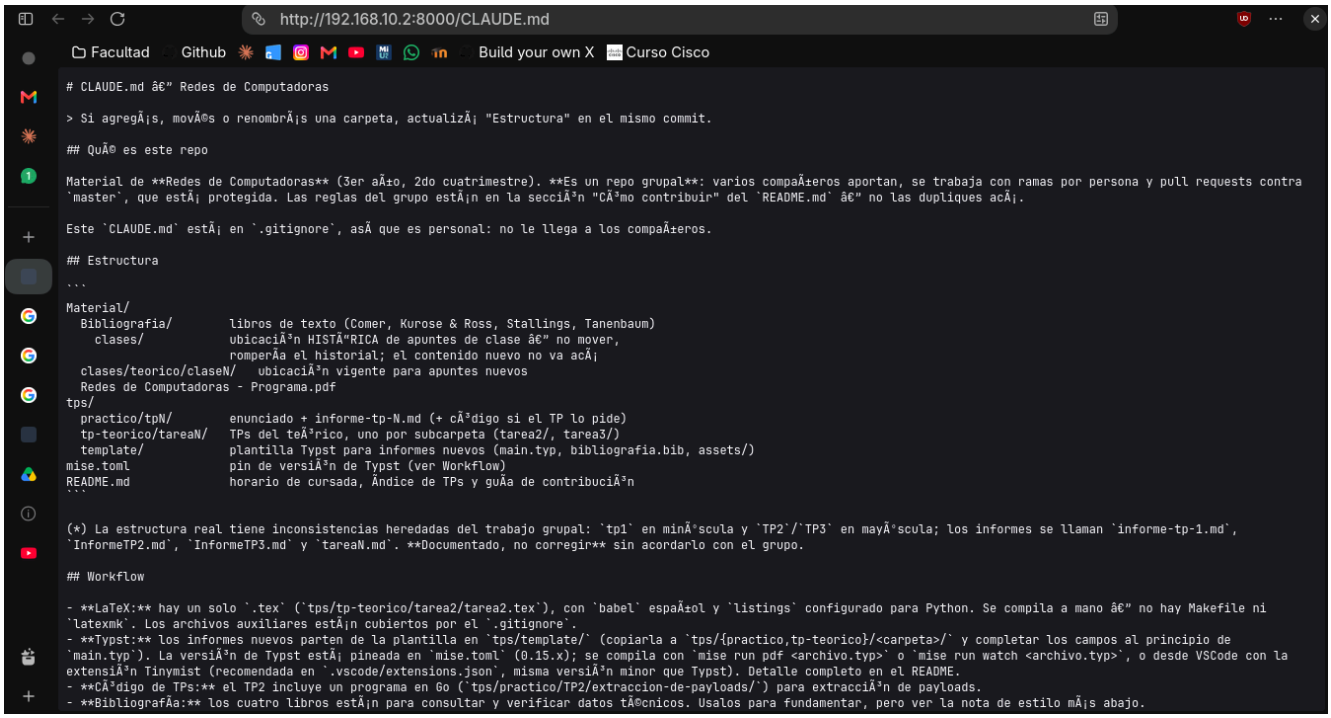


Figura 6. Archivo CLAUDE.md por la PC A vistos desde el navegador de la PC B.

En las capturas se ve que los acentos aparecen rotos (`Ã¡` en lugar de `á`, `â€"` en lugar de –). El servidor responde `Content-type: text/markdown` sin indicar `charset`, entonces el navegador no sabe que el archivo está en UTF-8 y lo interpreta como Latin-1. Cada carácter acentuado, que en UTF-8 ocupa dos bytes, se muestra como dos caracteres. El archivo llega bien; solo falla la visualización.

Consigna 3

Repetir la transferencia utilizando curl desde una terminal:

Repetimos las descargas desde la terminal de la PC B con `curl` (en las peticiones se ve `User-Agent: curl/8.22.0`):

1) `curl` a `http://192.168.10.2:8000/README.md` → `200 OK`, 6337 bytes de tipo `text/markdown` (`captura-http-readme.pcapng`).
2) Intentamos bajar el PDF del programa pidiendo `/Material/RedesdeComputadoras-Programa.pdf` → `404 File not found` (`captura-http-404-y-clase2.pcapng`). El archivo real se llama `Redes de Computadoras - Programa.pdf`: tiene espacios, que en la URL se escriben como `%20`. El servidor respondió igual, pero con una página HTML de error.
3) Bajamos un apunte de texto, `/Material/clases/teorico/clase2/clase2.txt` → `200 OK`, 63 424 bytes de tipo `text/plain` (`captura-http-404-y-clase2.pcapng`).

Consigna 4

Capturar el tráfico generado con Wireshark. ¿De qué tipo es?

Es tráfico **HTTP/1.0 sobre TCP**, puerto 8000, sobre IPv4 y Ethernet. A diferencia de la Parte 2, es **orientado a conexión y confiable**. Además viaja **en texto plano**: en Wireshark se leen completos los encabezados y el contenido del archivo, porque no hay cifrado (no es HTTPS).

La descarga de `README.md` (`captura-http-readme.pcapng`) muestra las tres fases de una conexión TCP:

Tabla 4. Descarga de `README.md` con `curl`, segmento a segmento.

Trama	Sentido	Segmento	Fase
19	B → A	`SYN` (MSS=1460, SACK, WS)	Establecimiento
20	A → B	`SYN, ACK`	Establecimiento
21	B → A	`ACK`	Establecimiento
22	B → A	`GET /README.md HTTP/1.1`	Petición HTTP
23	A → B	`ACK` de la petición	Transferencia
24	A → B	`HTTP/1.0 200 OK` (encabezados, 191 B)	Transferencia
26–34	A → B	Cuerpo en 4 segmentos de 1448 B y uno final de 545 B	Transferencia
25–33	B → A	Un `ACK` por cada segmento recibido	Transferencia
35	A → B	`FIN, ACK`	Cierre
36	B → A	`FIN, ACK`	Cierre
37	A → B	`ACK`	Cierre

Lo que se observa:

- **Establecimiento:** antes de cualquier dato hay un *three-way handshake*. Ahí se negocian el MSS (1460 bytes), SACK y el escalado de ventana. Esto es justo lo que no aparecía con UDP.
- **Segmentación:** el archivo se parte en segmentos de 1448 bytes de datos. Son los 1460 del MSS menos 12 bytes de la opción *timestamps* de TCP, y cada trama queda en 1514 bytes (el máximo de Ethernet sin FCS).
- **Confirmaciones:** el cliente confirma cada segmento con un ACK acumulativo (`Ack=1640`, `3088`, `4536`…). En la descarga más grande (`clase2.txt`, 63 424 bytes, cuya respuesta viajó en 45 segmentos) el cliente pasa a confirmar cada dos segmentos (*delayed ACK*).
- **Cierre por el servidor:** como el servidor responde con HTTP/1.0, cierra la conexión después de cada respuesta (envía el primer `FIN`). En la descarga de `clase2.txt` y en la del 404, el `FIN` viaja en el mismo segmento que los últimos datos. Cada archivo requiere una conexión TCP nueva.
- **Velocidad:** la transferencia completa de `README.md`, con *handshake* y cierre, tardó unos 13 ms. La de `clase2.txt` tardó 11 ms.

Las capturas también muestran tráfico ajeno a HTTP: el STP, LOOP, DTP y CDP del switch (como en la Parte 2), ARP entre las dos PCs, solicitudes ARP sin respuesta al gateway `192.168.10.1` (ver Consigna 1 d) e ICMPv6 *Router Solicitation* de una PC buscando un router IPv6 que no existe.

A. Un apéndice

Bibliografía

[1] W. Stallings, *Comunicaciones y redes de computadores*, 7.ª ed. Madrid: Pearson Educación, 2004.